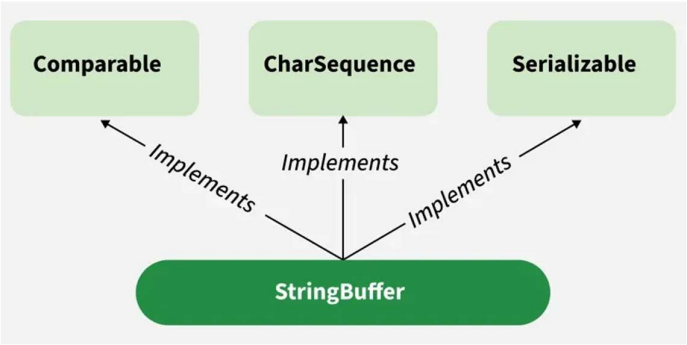
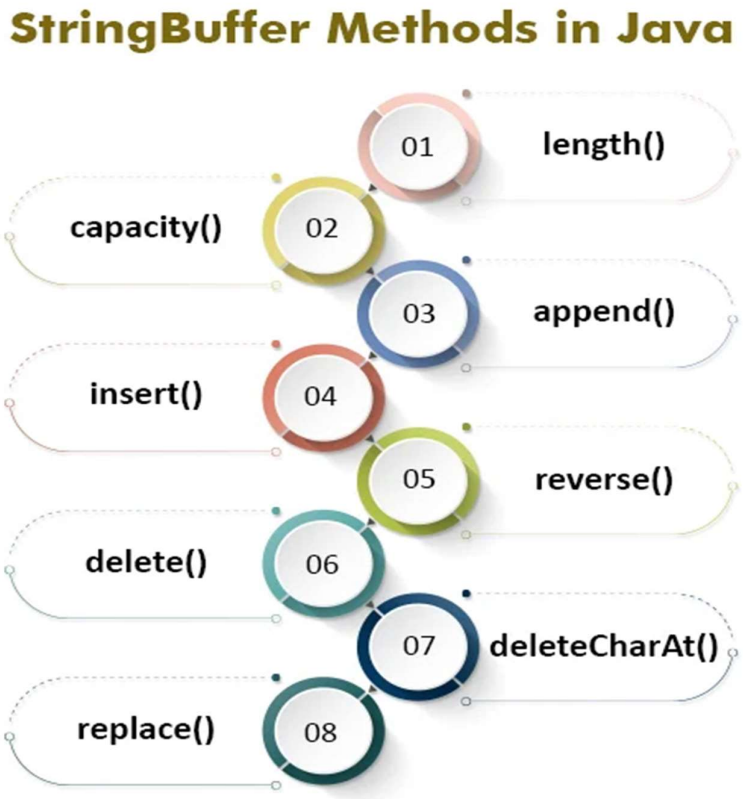
# String Buffer and String Builder

## String Buffer :-

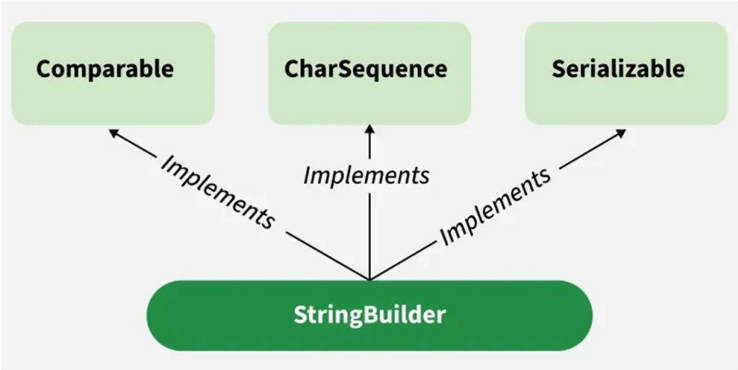



- **Syntax :-**

```
StringBuffer str = new StringBuffer();
```



## String Builder :-




- **Syntax :-**

**StringBuilder str = new StringBuilder();**

```
StringBuilder
    append(Object) : StringBuilder
    append(String) : StringBuilder
    append(StringBuffer) : StringBuilder
    append(CharSequence) : StringBuilder
    append(CharSequence, int, int) : StringBuilder
    append(char[]) : StringBuilder
    append(char[], int, int) : StringBuilder
    append(boolean) : StringBuilder
    append(char) : StringBuilder
    append(int) : StringBuilder
    append(long) : StringBuilder
    append(float) : StringBuilder
    append(double) : StringBuilder
    appendCodePoint(int) : StringBuilder
```

# Implementation of StringBuffer and StringBuilder

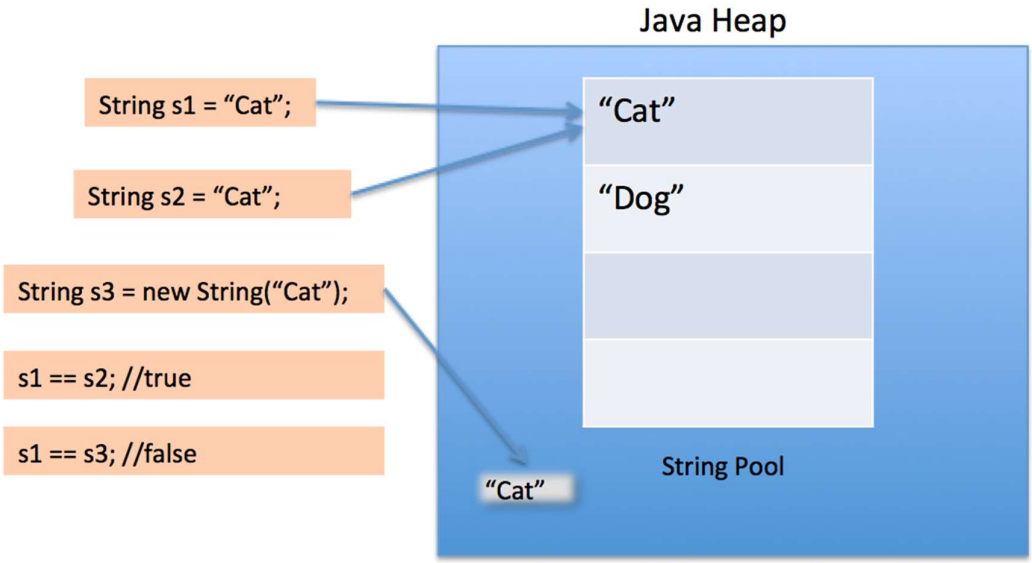
## Input :-

```java
package basicconcepts;

public class string_builderandbuffer {

    public static void main(String[] args) {

        System.out.println("_________________String Builder_________________________");

        StringBuilder str1=new StringBuilder("good");
        System.out.println("Before append: " +str1);
        System.out.println("Using append: " +str1.append("Student"));
        System.out.println("After append: "+str1);
        System.out.println("After reverse: " +str1.reverse());
        System.out.println("index of: "+str1.indexOf("o"));
        System.out.println("Charat: " +str1.charAt(0));
        System.out.println("Index of substring: " +str1.indexOf("t",1));
        System.out.println(str1.insert(0, "Nikhil"));
        str1.delete(0, 6);
        System.out.println("String is: " +str1);
        System.out.println(str1.repeat(str1, 8));

        System.out.println("_________________String Buffer_________________________");

        StringBuffer str2=new StringBuffer("bad");
        System.out.println("Before append: " +str2);
        System.out.println("Using append: " +str2.append("Student"));
        System.out.println("After append: "+str2);

    }
}
```

## Output :-

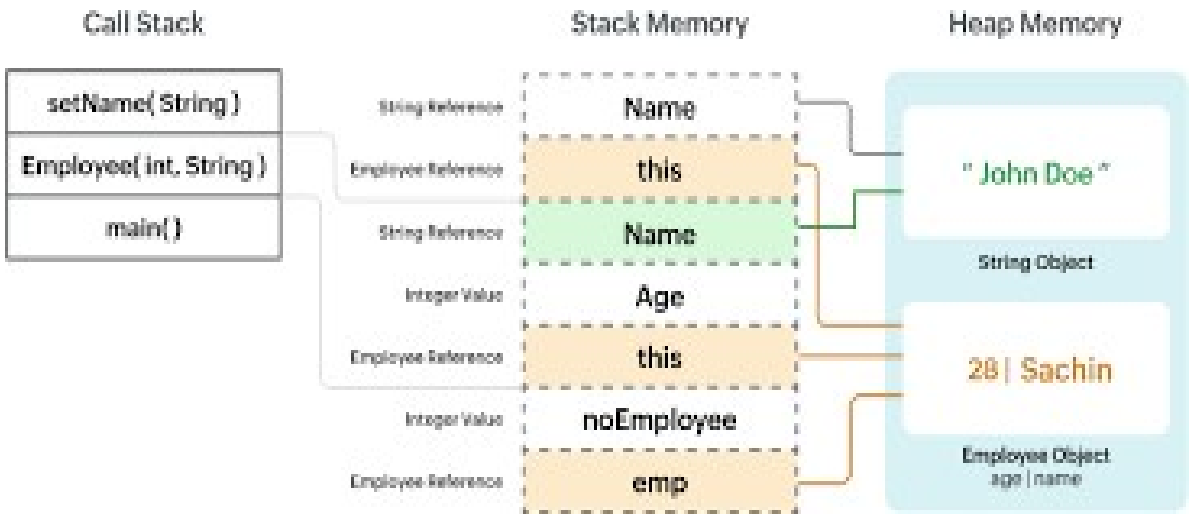
```
_________________String Builder_________________________
Before append: good
Using append: goodStudent
After append: goodStudent
After reverse: tnedutSdoog
index of: 8
Charat: t
Index of substring: 5
NikhiltnedutSdoog
String is: tnedutSdoog
tnedutSdoogtnedutSdoogtnedutSdoogtnedutSdoogtnedutSdoogtnedutSdoogtnedutSdoogtnedutSdoogtnedutSdoog
_________________String Buffer_________________________
Before append: bad
Using append: badStudent
After append: badStudent
```

# String Constant Pool and Heap




## Difference Between SCP and HeapArea :-

| Point | SCP (String Constant Pool) | Heap Area |
|---|---|---|
| **1. Location** | Special part inside Heap memory | Main memory area of JVM |
| **2. What it stores** | Only string literals | All objects, arrays, wrapper classes, etc. |
| **3. Memory optimization** | Avoids duplicates by reusing same string reference | Allows duplicates, every new creates a new object |
| **4. Type of objects** | Only immutable String objects | Both mutable (e.g., StringBuilder) and immutable objects |
| **5. Garbage Collection** | Managed specially by JVM, rarely collected | Normal garbage collection by JVM |




**Difference Between String, StringBuffer and StringBuilder :-**

| **Feature** | **String (Immutable)** | **StringBuffer (Mutable + Thread-safe)** | **StringBuilder (Mutable + Fast)** |
|---|---|---|---|
| Mutability | Immutable (cannot change) | Mutable (can change) | Mutable (can change) |
| Thread-safety | Safe (due to immutability) | Thread-safe (synchronized) | Not thread-safe |
| Performance | Slow if many modifications | Slower than StringBuilder | Faster than StringBuffer |
| Object creation | New object created on every change | Modifies same object | Modifies same object |
| Best use case | Fixed text or fewer changes | Multi-threaded environment | Single-threaded environment |